

Faculty of Electrical Engineering
Banja Luka
Programming languages 2

Project Terms of Reference

May 2024.

ePJ2 is **an e-mobility** company that rents electric cars, bicycles and scooters, in the narrow and wider area of the city of Java. The goal of the project assignment is to create a program that will simulate the use of means of transport based on predefined data and generate detailed financial calculations, statistics and monitor the condition of all means of transport used.

The company has at its disposal a certain number of cars for which the following data is stored: unique identifier (ID), purchase date, purchase price, manufacturer, model, description, current battery level. The following data is stored for electric bicycles: unique identifier (ID), manufacturer, model, purchase price, current battery level, range on a single charge (autonomy). The following data is stored for electric scooters: unique identifier (ID), manufacturer, model, current battery level, purchase price and maximum speed. All means of transport can break down, and the reason for the breakdown (description) and the date and time are recorded. All vehicles have the ability to charge the battery (a method that increases the current level).

The battery is discharged while moving. The car has the ability to transport more people.

The main business of the company is the rental of means of transport. When renting, the date and time of the rental, the name of the user, the current location where the vehicle is picked up, the location where the vehicle is left after use and the duration of use in seconds are recorded. When renting a car, it is necessary to submit an identification document (passport for foreign citizens and identity card for domestic citizens) and driver's license (number). Based on this data, a payment invoice is generated and delivered to users as **a txt** file.

The company defines the rental prices of each type of means of transport by the time of use in seconds. Based on the duration of the drive and additional factors, the total price to be paid is calculated. Additional factors that affect the price are the use of the means of transport in the wider part of the city, breakdowns, discounts and promotions, and the method of calculation is given in the table. All parameters are stored outside the program in **properties** files (<https://www.baeldung.com/java-properties>).

Description	Formula (parameters stored in prop. files are in capital letters)
Distance	Basic price * DISTANCE_NARROW for the narrowest part of the city, basic price * DISTANCE_WIDE for the wider part of the city. If at least one path field is in the wider part of the city, then the tariff for the wider part of the city (DISTANCE_WIDE) is applied.
Malfunctions	Base price = 0, only if a failure occurred
Discount	For every 10th rental: Base price * DISCOUNT (%)
Promotions	Basic price * DISCOUNT_PROM (%) for rentals where there was a promotion
Basic price For cars:	CAR_UNIT_PRICE * seconds of driving time

	For bicycles: BIKE_UNIT_PRICE * seconds of ride duration For scooters: SCOOTER_UNIT_PRICE * seconds of driving time
--	--

Calculation example:

Amount (base price * distance) = ((SCOOTER_UNIT_PRICE * Duration) *
 DISTANCE_WIDE)
 Total to pay(Amount - discount - promotion) = Amount - (DISCOUNT * Amount) -
 (DISCOUNT_PROM * Amount)

All items should be listed individually on invoices. If there is a failure then Total to Pay = 0.

All information about means of transport and rentals will be available on the course's Moodle page. For each defense of the project tasks, the test data can be changed, so nothing must be defined in the code.

Rental data is used to simulate the company's operations. The data file is loaded one row at a time, sorted by rental time, and that data is used to display map movement in separate threads for each row. It is mandatory that the simulation is performed according to the rental order (date and time). One thread is bound to only one rental, and the map is common to all threads. The simulation is performed for as many seconds as the rental duration is defined. As the starting and ending points of the rental are known, students should create a direct path between them and thus move the display on the map.

The display can be implemented so that the map fields are marked with a different color and the vehicle identifier and battery level are printed. When the simulation of all rentals for a certain date and time is finished, a pause of 5 seconds should be made and then the rental simulations for the next date and time should start. When simulating one rental is finished, an invoice is created that has all the data related to that rental and the total price to pay. In addition, it is necessary to specify items related to discounts or promotions, if any. Files are saved in a folder on a specific path (parameter in *the properties* file).

The program should have several graphical interfaces for displaying data that can be implemented as JavaFX or Swing GUI: - main screen

for displaying the map, - screen for
 displaying all means of transport (3 tables, print all information), - screen for displaying faults
 (1 table, columns: type of means of transport, ID, time, description
 failure)
 and - screen for displaying business results.

The map consists of 20x20 squares. The fields marked in white in the picture (outside) are the wider part of the city, and the fields marked in blue are the narrower part of the city. Field 0,0 is the top left first, and field 19,19 is the bottom right. Test data is defined in this way. Permitted paths are exclusively straight, and the duration of the simulation is divided by the number of fields that the vehicle will cross on the map, and thus the time spent on one field is defined. The positions of the means of transport on the map are displayed in real time.

Business results should show all income (amounts from generated accounts) and all additional parameters important for business. There are 2 types of reports, summary and daily. For the summary report, it is necessary to print the following data:

1. total income (sum of all payment amounts on all accounts), 2. total discount (sum of all discount amounts from all accounts), 3. total promotions (sum of the value of all promotions from all accounts), 4. total amount of all rides in the narrow and wider part of the city, 5. total amount for maintenance (total income * 0.2), 6. total amount for breakdown repairs (coefficient (cars 0.07, bicycles 0.04, scooters 0.02) * purchase price of a broken down means of transport). It is calculated as the sum of all individual failures. 7. total company expenses (20% of total revenue) and 8. total tax (total revenue - total amount for maintenance - total amount for repair of breakdowns - company costs) * 10%.

Daily reports show items 1-6 grouped by date. Show all available dates.

This report is in tabular form.

Depending on the number of indexes, it is necessary to implement one additional functionality, out of the 3 listed (e.g. $1234/24 \Rightarrow$ sum of digits is $1+2+3+4+2+4=16$, $(16\%3 + 1)=2$):

1. find the means of transport that brought the most revenue for each type, 2. find the means of transport that brought the most losses for each type, 3. find the means of transport that broke down and the cost of repairs.

This data is individually serialized into binary files, where the vehicle object with all fields is saved and exported to a specific folder. It is necessary to implement a screen on which this data can be deserialized and displayed.

Map image:

[illegible]

Notes: -

- this project task is valid until the publication of a new project task, - it is mandatory to add JavaDoc comments and generate documentation, -
- it is mandatory to create packages, - avoid code duplication and - take care of code quality (names of classes, methods, variables, alignments, performance...).